

Datos de la API en gráficas

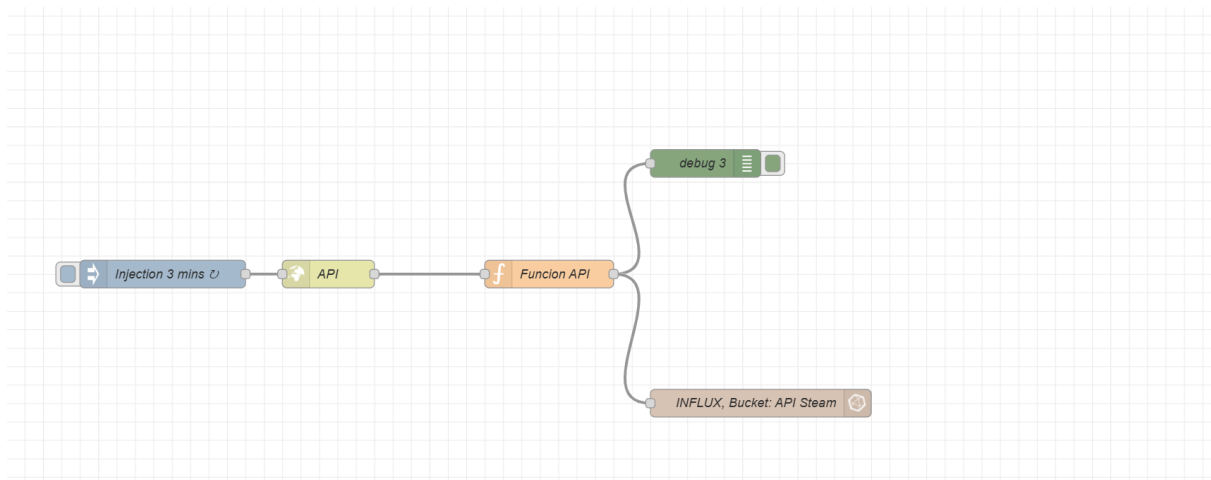


En este documento se presenta el trabajo realizado por el grupo 2 en Node-RED, InfluxDB y Grafana en distintos apartados explicados. Los datos de la API que se visualizan en las gráficas son sacados de la fuente de datos de [Steam](#) y la que usamos en este grafico en concreto es la del videojuego [CSGO2](#) con la ID 730. En este caso representa la cantidad de jugadores que hay en distintas fechas y horas.

ÍNDICE

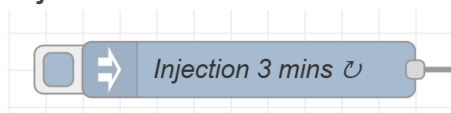
<u>Node-RED 3</u>	3
Injection:.....	3
API:.....	3
Función API:.....	4
Debug:.....	4
Influx out:.....	4
<u>InfluxDB 5</u>	5
<u>Grafana 6</u>	6

Node-RED



La foto muestra la estructura principal que se ha montado en Node-RED, la cual esta compuesta por:

Injection:



La función injection tiene configurado un intervalo de 3 mins

Editar nodo inject

Eliminar Cancelar Hecho

Propiedades

Nombre: Injection 3 mins

msg.payload: milisegundos desde epoch

msg.topic: 1

+ añadir

☐ Inyectar una vez después de 0.1 segundos, entonces

Repetir: intervalo

cada 3 minutos

API:



El bloque API se encarga de la api que conecta la estructura con los datos. En este caso hemos usado la siguiente api:

["https://api.steampowered.com/ISteamUserStats/GetNumberOfCurrentPlayers/v1/?appid=730"](https://api.steampowered.com/ISteamUserStats/GetNumberOfCurrentPlayers/v1/?appid=730)

Es una api recopila los datos constantes de steam y en este caso concretamente de CSGO2 ya que tiene la aptoide=730 al final. Si se necesita visualizar los datos de otros videojuegos es tan fácil como poner al final la ID del videojuego concreto.

En la imagen de la derecha se puede ver como está creado el bloque, importante poner la opción "devolver" como "un objeto JSON analizado".

Editar nodo http request

Eliminar Cancelar Hecho

Propiedades

Método: GET

URL: https://api.steampowered.com/ISteamUserStats/G

Carga: Ignorar

☐ Usar configuración SSL/TLS personalizada

☐ Usar autenticación

☐ Habilitar conexión activa (keep-alive)

☐ Usar proxy

☐ Enviar solo respuestas que no sean 2xx al nodo Catch

☐ Deshabilitar el análisis estricto de HTTP

Devolver: un objeto JSON analizado

Tip: If the JSON parse fails the fetched string is returned as-is.

Función API:



Se encarga de sacar el numero de jugadores que hay de ese juego con una actualización constante de entre 3 y 5 minutos. Descarta los errores y cuando este preparado lo envia a influxDB, dentro del bloque se encuentra el siguiente código:

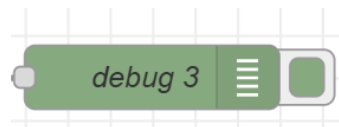
```
const resp = msg.payload && msg.payload.response;
const jugadores = resp ? resp.player_count : undefined;

if (typeof jugadores !== "number") {
  node.warn("Respuesta inesperada de Steam: " +
    JSON.stringify(msg.payload));
  node.status({ fill: "red", shape: "ring", text: "Sin dato" });
  return null;
}

msg.payload = { jugadores: jugadores };

node.status({ fill: "green", shape: "dot", text: "Jugadores: " +
  jugadores });
return msg;
```

Debug:



El debug esta para enseñar en la barra de la derecha los datos que se pasan hacia influxDB

Influx out:



El influx out se usa para conectar los datos con influxDB, la configuración al uso en el proyecto es la siguiente:

Editar nodo influxdb out

Eliminar Cancelar Hecho

Propiedades

Nombre INFLUX, Bucket: API Steam

Servidor [v2.0] http://influxdb:8086

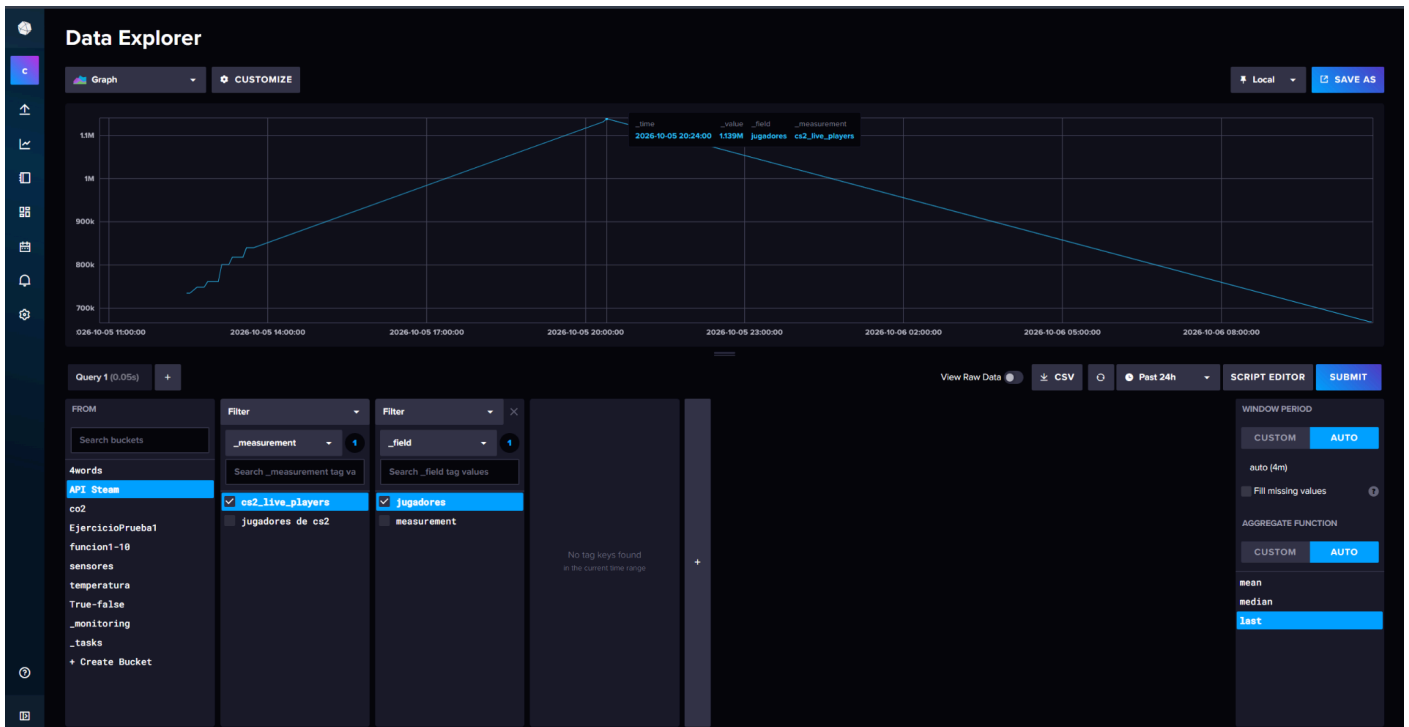
Organización clase

Bucket API Steam

Medición cs2_live_players

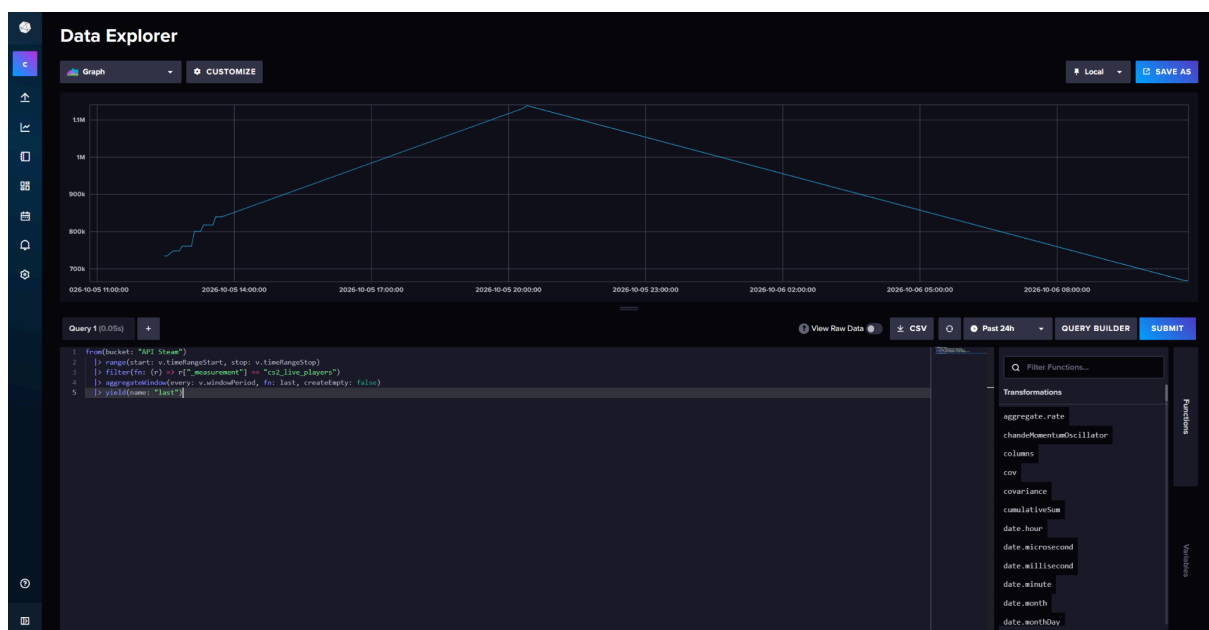
Precisión Milliseconds (ms)

InfluxDB



En influx recibimos los datos de Node-RED y con ellos hemos hecho un bucket que se llama API Steam con el measurement “CS2 _live_players” y con el field “jugadores”. Cuando le damos a submit podemos ver un primer vistazo gráfico en líneas de los datos recibidos de la API.

Para sacar el script para grafana simplemente hay que darle al botón “script editor” al lado de –Tsubmit:



Grafana

En grafana hemos creado un dashboard nuevo en forma de panel gráfico de líneas:



En el vemos como escala y baja el pico de jugadores en las diferentes horas y fechas con los jugadores a la izquierda representados con números.

Organigrama

